The Lumen Compiler Handbook

A ground-up, project-based path through compiler engineering, in C++

How this handbook works

This is a **living document**. Phases 0–1 below are complete: full theory, internal design, working C++, exercises, and real-compiler comparison. Phases 2–9 are complete in *theory and design* — every concept, trade-off, and decision a real compiler engineer weighs — but their C++ implementations get added incrementally as we build them together, so the code always comes with the understanding behind it rather than being pasted in cold. Each session, we extend this same file. Nothing here is throwaway.

The pipeline we're building (project name: **Lumen**):

```
source (.lum) → Lexer → Tokens → Parser → AST → Sema → typed AST
→ IRGen → IR (three-address code / SSA) → Optimizer → optimized IR
→ Codegen → x86-64 asm → Assembler/Linker (system tools) → executable
```

Phase 0 — Foundations

0.1 What a compiler is

A compiler translates source code into another representation — usually one closer to what a machine executes. The translation happens in stages because each stage answers a different question:

Stage	Question
Lexing	What are the atomic words in this text?
Parsing	Do these words form valid grammatical structure?
Semantic analysis	Does this valid structure make sense (types, scope)?
IR generation	What does this program *mean*, independent of any CPU?
Optimization	How do we express that meaning more efficiently?
Codegen	How do we turn that meaning into instructions for *this* CPU?

Separating these stages is why GCC supports 30+ languages and 50+ targets without an N×M explosion of translators: every frontend lowers to a shared IR, every backend only consumes that IR. Frontend and backend don't know about each other.

Compiler vs interpreter vs JIT: a compiler produces a translated artifact ahead of time, then stops. An interpreter executes source (or an IR of it) directly, no separate artifact. A JIT

starts by interpreting, then compiles hot paths to native code at runtime (JVM, V8, LuaJIT). Lumen is the classic ahead-of-time kind.

0.2 The pipeline as data transformations

```
"int x = 2 + 3;"
      | Lexer
      ▼
[INT, IDENT(x), EQUALS, NUM(2), PLUS, NUM(3), SEMI]
      | Parser
      ▼
VarDecl(x, int, BinaryExpr(+, 2, 3))            ← AST
      | Sema (annotates/checks in place)
      ▼
VarDecl(x, int, BinaryExpr(+, 2, 3))            ← now type-checked
      | IRGen
      ▼
t1 = 2 + 3
x  = t1
      | Codegen
      ▼
mov eax, 5
mov [x], eax
```

Each arrow is a **pass**: a pure-ish function taking one representation to the next. Lumen's `main()` will eventually just be this pipeline, each stage independently testable.

0.3 Project skeleton

```
lumen/
├── CMakeLists.txt
├── src/
│   ├── main.cpp
│   ├── lexer/
│   ├── parser/
│   ├── ast/
│   ├── sema/
│   ├── ir/
│   └── codegen/
├── tests/
└── examples/
```

`CMakeLists.txt` (C++20, no dependencies yet):

```
cmake
```

```
cmake_minimum_required(VERSION 3.20)
project(lumen CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(lumen
    src/main.cpp
)
target_include_directories(lumen PRIVATE src)
```

`src/main.cpp`:

```cpp
```

```cpp
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

// Reads an entire file into memory as one string.
// Why read it all at once rather than stream character-by-character?
// A compiler needs random-ish lookahead/backtracking during lexing and
// parsing (e.g. peeking multiple tokens ahead), and source files are
// small relative to available RAM (megabytes, not gigabytes) — so the
// simplicity of "whole file in memory" wins. Real compilers *do* stream
// for huge generated files or use memory-mapped I/O, but that's an
// optimization layered on top of this same logical model, not a
// different design.
static std::string readFile(const std::string& path) {
    std::ifstream file(path);
    if (!file) {
        throw std::runtime_error("could not open file: " + path);
    }
    std::ostringstream contents;
    contents << file.rdbuf();
    return contents.str();
}

int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "usage: lumen <source-file>\n";
        return 1;
    }
    try {
        std::string source = readFile(argv[1]);
        std::cout << source;
    } catch (const std::exception& e) {
        std::cerr << "error: " << e.what() << "\n";
        return 1;
    }
    return 0;
}
```

Exercise 0.1: Build this, run it on a small `.lum` file. Confirm it round-trips the text.

Exercise 0.2 (think, don't code): What breaks in this "read it all up front" model if Lumen later needs to support `#include`-style file composition? Sketch how you'd extend it.

Phase 1 — Lexical Analysis

1.1 Theory

The lexer (scanner) turns a flat stream of characters into a stream of **tokens** — the smallest meaningful units: keywords, identifiers, literals, operators, punctuation. `int x = 2 + 3;` becomes `[INT, IDENT("x"), EQUALS, NUMBER(2), PLUS, NUMBER(3), SEMI]`.

Why not let the parser work directly on characters? Two reasons:

1. **Separation of concerns** — "what is this word" and "how do words combine grammatically" are different problems with different tools (regular languages vs context-free languages).
2. **Efficiency and simplicity** — grammars that mix character-level and structure-level reasoning become unreadable. Splitting lets each stage use the simplest adequate formalism.

The formal foundation: tokens are defined by **regular expressions**, and regular expressions are equivalent to **finite automata** (Kleene's theorem). Every token rule — "an identifier is a letter followed by letters/digits" — is a regular language, recognizable by a DFA (deterministic finite automaton) that reads one character at a time and moves between states, accepting when it lands in an accept state at end-of-token.

`IDENT` as an NFA (nondeterministic, easy to build from the regex) is converted to a DFA (deterministic, easy to execute in O(1) per character — no backtracking) via the subset construction (Thompson's construction → NFA, then powerset construction → DFA). Tools like `flex` do exactly this at generation time. **Real compilers almost never do this at generation time via a tool** for hand-tuned frontends — Clang and GCC hand-write their lexers as a big switch/state machine, because a hand-written lexer can special-case common patterns (identifiers, whitespace) faster than a generated table-driven one, and because it integrates error recovery and diagnostics far better. We'll hand-write Lumen's lexer too — it's also pedagogically clearer.

1.2 Internal working

Data structures:

- `Token`: kind (enum), lexeme (string or view into source), and source position (line/column) for diagnostics.
- `Lexer`: holds the source string, a cursor index, and line/column counters. It exposes `nextToken()`, called repeatedly by the parser (or once up front to produce a `vector<Token>` — we'll do the latter for simplicity, real compilers often do the former to avoid materializing all tokens).

Algorithm shape (maximal munch): at each position, try to match the *longest* possible token. `<=` must be recognized as one token, not `<` then `=`. This is why the lexer peeks ahead before committing to a token kind.

Complexity: O(n) in source length — each character is examined a constant number of times (this is exactly what a DFA guarantees; a hand-written lexer preserves this if written

carefully, i.e., no re-scanning from the start on failure).

Design decisions we're making for Lumen (and why):

- Store lexemes as `std::string_view` into the original source buffer (which we keep alive for the whole compilation) rather than copying substrings — avoids O(n) copies, matches what Clang does (`llvm::StringRef`).
- Track line/column incrementally during scanning rather than recomputing later — diagnostics are cheap this way and it costs nothing extra since we're already walking the text.
- Fail *softly*: an illegal character produces an `ERROR` token with a message, not a thrown exception, so the lexer can keep going and the parser can eventually report multiple errors in one pass — this is what "error recovery" starts with.

1.3 C++ implementation

`src/lexer/token.hpp`:

cpp

```cpp
#pragma once
#include <string_view>

namespace lumen {

enum class TokenKind {
    // literals & identifiers
    Identifier, Number,
    // keywords
    KwInt, KwIf, KwElse, KwWhile, KwReturn,
    // punctuation & operators
    Plus, Minus, Star, Slash,
    Equals, EqualsEquals, Less, LessEqual, Greater, GreaterEqual,
    LParen, RParen, LBrace, RBrace, Semicolon,
    // control
    EndOfFile, Error
};

struct Token {
    TokenKind kind;
    std::string_view lexeme;   // view into the source buffer — no copy
    int line;
    int column;
};

} // namespace lumen
```

`src/lexer/lexer.hpp` / `.cpp`:

cpp

```cpp
#pragma once
#include <string>
#include <vector>
#include "token.hpp"

namespace lumen {

class Lexer {
public:
    explicit Lexer(const std::string& source) : src_(source) {}

    // Scans the whole source into a token vector, ending with EndOfFile.
    std::vector<Token> tokenize();

private:
    const std::string& src_;
    size_t pos_ = 0;
    int line_ = 1;
    int column_ = 1;

    bool atEnd() const { return pos_ >= src_.size(); }
    char peek() const { return atEnd() ? '\0' : src_[pos_]; }
    char peekNext() const { return pos_ + 1 < src_.size() ? src_[pos_ + 1] : '\
    char advance();                // consumes and returns current char, updates
    bool match(char expected);     // consumes current char IF it equals expecte

    void skipWhitespaceAndComments();
    Token makeToken(TokenKind kind, size_t start, int startLine, int startColum
    Token errorToken(const std::string& msg, int startLine, int startColumn);

    Token scanIdentifierOrKeyword();
    Token scanNumber();
    Token scanOperatorOrPunctuation();

    static TokenKind keywordOrIdentifier(std::string_view text);
};

} // namespace lumen
```

cpp

```
#include "lexer.hpp"
#include <cctype>
#include <unordered_map>

namespace lumen {

char Lexer::advance() {
    char c = src_[pos_++];
    if (c == '\n') { line_++; column_ = 1; } else { column_++; }
    return c;
}

bool Lexer::match(char expected) {
    if (atEnd() || src_[pos_] != expected) return false;
    advance();
    return true;
}

void Lexer::skipWhitespaceAndComments() {
    while (!atEnd()) {
        char c = peek();
        if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
            advance();
        } else if (c == '/' && peekNext() == '/') {
            while (!atEnd() && peek() != '\n') advance();
        } else {
            return;
        }
    }
}

Token Lexer::makeToken(TokenKind kind, size_t start, int startLine, int startCo
    std::string_view lexeme(src_.data() + start, pos_ - start);
    return Token{kind, lexeme, startLine, startColumn};
}

Token Lexer::errorToken(const std::string& msg, int startLine, int startColumn)
    // In a fuller implementation this would carry `msg` in a diagnostics
    // side-channel rather than the lexeme; kept simple here.
    return Token{TokenKind::Error, std::string_view(msg), startLine, startColum
}

TokenKind Lexer::keywordOrIdentifier(std::string_view text) {
    static const std::unordered_map<std::string_view, TokenKind> keywords = {
        {"int", TokenKind::KwInt}, {"if", TokenKind::KwIf},
        {"else", TokenKind::KwElse}, {"while", TokenKind::KwWhile},
```

```
        {"return", TokenKind::KwReturn},
    };
    auto it = keywords.find(text);
    return it != keywords.end() ? it->second : TokenKind::Identifier;
}

Token Lexer::scanIdentifierOrKeyword() {
    size_t start = pos_; int line = line_, col = column_;
    while (!atEnd() && (std::isalnum((unsigned char)peek()) || peek() == '_'))
    std::string_view text(src_.data() + start, pos_ - start);
    return Token{keywordOrIdentifier(text), text, line, col};
}

Token Lexer::scanNumber() {
    size_t start = pos_; int line = line_, col = column_;
    while (!atEnd() && std::isdigit((unsigned char)peek())) advance();
    // (Deliberately no floats/decimals yet — Lumen starts int-only; this
    // is where you'd extend for '.', exponents, etc.)
    return makeToken(TokenKind::Number, start, line, col);
}

Token Lexer::scanOperatorOrPunctuation() {
    size_t start = pos_; int line = line_, col = column_;
    char c = advance();
    switch (c) {
        case '+': return makeToken(TokenKind::Plus, start, line, col);
        case '-': return makeToken(TokenKind::Minus, start, line, col);
        case '*': return makeToken(TokenKind::Star, start, line, col);
        case '/': return makeToken(TokenKind::Slash, start, line, col);
        case '(': return makeToken(TokenKind::LParen, start, line, col);
        case ')': return makeToken(TokenKind::RParen, start, line, col);
        case '{': return makeToken(TokenKind::LBrace, start, line, col);
        case '}': return makeToken(TokenKind::RBrace, start, line, col);
        case ';': return makeToken(TokenKind::Semicolon, start, line, col);
        case '=':
            // maximal munch: check for '==' before committing to '='
            if (match('=')) return makeToken(TokenKind::EqualsEquals, start, li
            return makeToken(TokenKind::Equals, start, line, col);
        case '<':
            if (match('=')) return makeToken(TokenKind::LessEqual, start, line,
            return makeToken(TokenKind::Less, start, line, col);
        case '>':
            if (match('=')) return makeToken(TokenKind::GreaterEqual, start, li
            return makeToken(TokenKind::Greater, start, line, col);
        default:
            return errorToken(std::string("unexpected character '") + c + "'",
    }
```

```
}

std::vector<Token> Lexer::tokenize() {
    std::vector<Token> tokens;
    while (true) {
        skipWhitespaceAndComments();
        if (atEnd()) {
            tokens.push_back(Token{TokenKind::EndOfFile, {}, line_, column_});
            break;
        }
        char c = peek();
        if (std::isalpha((unsigned char)c) || c == '_') {
            tokens.push_back(scanIdentifierOrKeyword());
        } else if (std::isdigit((unsigned char)c)) {
            tokens.push_back(scanNumber());
        } else {
            tokens.push_back(scanOperatorOrPunctuation());
        }
    }
    return tokens;
}

} // namespace lumen
```

Walkthrough of the design choices actually made here:

- `tokenize()` is a **greedy loop**, not recursion — lexers don't need call-stack depth proportional to input, so an explicit loop is both simpler and avoids any risk of stack overflow on huge files.
- Every `scan*` function records `start/line/col` *before* consuming, so the resulting token's position points at its first character — essential for diagnostics like `error at line 4, column 9`.
- `match()` is the mechanism for maximal munch: we optimistically consume the "long" form only if the next character confirms it.

1.4 Exercises

1. Add float literal support (`3.14`) to `scanNumber`.
2. Add `&&`, `||`, `!` — decide maximal-munch handling for `!` vs `!=`.
3. **Debugging exercise:** the current lexer has no way to signal "line 4 col 9: unexpected character '@'" back to `main`. Design (don't necessarily implement yet) a `DiagnosticEngine` that the lexer reports into instead of encoding errors in tokens.
4. **Interview-style:** why is maximal munch sometimes *wrong* in real languages? (Hint: look up C++'s `>>` template-closing ambiguity, and how the C++11 standard changed lexing rules to fix it.)

5. Write 10 hand-crafted `.lum` snippets and their expected token streams; turn these into unit tests now, before the parser exists — this becomes your regression suite for the rest of the project.

1.5 Real compiler comparison

- **Clang**: hand-written lexer (`clang::Lexer`), operates lazily token-by-token, heavily optimized (branchless character classification tables), integrated with a `SourceManager` for precise diagnostics including macro-expansion history.
- **GCC**: also hand-written (`libcpp`), notably handles the C preprocessor *as part of* lexing, since `#define`/`#include` interact with tokenization itself.
- **rustc**: hand-written lexer producing a token stream consumed by a hand-written recursive-descent-ish parser; notably lexes raw strings, integer suffixes, and unicode identifiers as first-class concerns from day one.
- **Go compiler**: hand-written, deliberately minimal token set (Go has few keywords by design), reflecting the language's philosophy in the tool.

The common thread: production compilers overwhelmingly hand-write lexers rather than using `flex`/generator tools, precisely because of the error-recovery and diagnostic-quality reasons above — validating the approach we're taking with Lumen.

Phase 2 — Syntax Analysis

2.1 Theory

Where lexing handles *regular* languages, parsing handles **context-free languages** — grammars that can express nesting (`( ( ( x ) ) )`), which regular expressions fundamentally cannot (no finite automaton can count arbitrarily deep nesting). A **context-free grammar** (CFG) is a set of production rules like:

```
Expr   → Expr '+' Term | Term
Term   → Term '*' Factor | Factor
Factor → NUMBER | '(' Expr ')'
```

Parsing = given a token stream, find a derivation (or prove none exists) — i.e., build a **parse tree**. We then simplify the parse tree into an **AST**, which drops grammar scaffolding (parentheses, redundant unary productions) and keeps only semantically meaningful structure.

2.2 Parsing strategies — the real trade-off space

- **Recursive descent (top-down):** one function per grammar rule, calling each other following the grammar's structure. Simple to write, simple to add great error messages and recovery to (because you control the code, not a table). **Limitation:** naturally handles LL grammars only — left recursion (`Expr → Expr '+' Term`) causes infinite recursion and must be rewritten iteratively (this is exactly how Pratt/precedence-

climbing parsing handles arithmetic — we'll derive this together next session). **This is what Clang, and most hand-written production frontends, use.**

- **LL(1) table-driven:** formalizes recursive descent into a table; same expressive power, less flexible for custom error messages, mostly of pedagogical/generator-tool interest now.
- **LR/LALR (bottom-up, e.g. yacc/bison):** strictly more powerful (handles left recursion naturally, more grammars describable unambiguously), but the generated parser's control flow doesn't mirror the grammar the way recursive descent does, making custom diagnostics harder. Used historically (original C compilers), less common in new hand-written frontends today.

Decision for Lumen: recursive descent with precedence climbing for expressions — matches what Clang/rustc actually do and keeps error recovery in our hands.

2.3 Lumen's grammar

```
Program    → Function*
Function   → 'int' IDENT '(' ')' Block
Block      → '{' Stmt* '}'
Stmt       → VarDecl | ExprStmt | IfStmt | WhileStmt | ReturnStmt | Block
VarDecl    → 'int' IDENT ('=' Expr)? ';'
ExprStmt   → Expr ';'
IfStmt     → 'if' '(' Expr ')' Stmt ('else' Stmt)?
WhileStmt  → 'while' '(' Expr ')' Stmt
ReturnStmt → 'return' Expr? ';'
Expr       → Assignment
Assignment → IDENT '=' Assignment | Equality
Equality   → Comparison (('=='|'!=') Comparison)*
Comparison → Term (('<'|'<='|'>'|'>=') Term)*
Term       → Factor (('+'|'-') Factor)*
Factor     → Unary (('*'|'/') Unary)*
Unary      → '-' Unary | Primary
Primary    → NUMBER | IDENT | '(' Expr ')'
```

Notice `Term → Term '*' Factor` is *left*-recursive on paper, which would blow the stack if implemented as literal recursive descent. **Precedence climbing** rewrites this into a loop: parse one `Factor`, then while the next token is `+`/`-`, consume it and fold in another `Factor`. Same language, no recursion on the left, one level of the grammar per precedence tier.

2.4 AST node hierarchy

`src/ast/ast.hpp`:

cpp

```cpp
#pragma once
#include <memory>
#include <string>
#include <vector>
#include "../lexer/token.hpp"

namespace lumen {

// ---- Expressions ----
struct Expr { virtual ~Expr() = default; int line = 0; };
using ExprPtr = std::unique_ptr<Expr>;

struct NumberExpr : Expr { int value; };
struct VarExpr    : Expr { std::string name; };
struct AssignExpr : Expr { std::string name; ExprPtr value; };
struct BinaryExpr : Expr { TokenKind op; ExprPtr lhs, rhs; };
struct UnaryExpr  : Expr { TokenKind op; ExprPtr operand; };

// ---- Statements ----
struct Stmt { virtual ~Stmt() = default; int line = 0; };
using StmtPtr = std::unique_ptr<Stmt>;

struct VarDeclStmt : Stmt { std::string name; ExprPtr init; /* may be null */ }
struct ExprStmt    : Stmt { ExprPtr expr; };
struct BlockStmt   : Stmt { std::vector<StmtPtr> statements; };
struct IfStmt      : Stmt { ExprPtr cond; StmtPtr thenBranch; StmtPtr elseBranc
struct WhileStmt   : Stmt { ExprPtr cond; StmtPtr body; };
struct ReturnStmt  : Stmt { ExprPtr value; /* may be null */ };

struct FunctionDecl {
    std::string name;
    std::unique_ptr<BlockStmt> body;
};

struct Program {
    std::vector<FunctionDecl> functions;
};

} // namespace lumen
```

We use `unique_ptr` + raw ownership trees (not a visitor-pattern indirection layer yet) — simplest thing that works. When we add multiple passes over the same tree (sema, IR gen, and later a pretty-printer), *that's* the point where a visitor pattern earns its complexity; introducing it now would be premature abstraction.

2.5 The parser

`src/parser/parser.hpp` / `.cpp`:

```cpp
```

```cpp
#pragma once
#include <vector>
#include "../lexer/token.hpp"
#include "../ast/ast.hpp"

namespace lumen {

class Parser {
public:
    explicit Parser(std::vector<Token> tokens) : tokens_(std::move(tokens)) {}
    Program parseProgram();

private:
    std::vector<Token> tokens_;
    size_t pos_ = 0;

    const Token& peek() const { return tokens_[pos_]; }
    const Token& previous() const { return tokens_[pos_ - 1]; }
    bool check(TokenKind k) const { return peek().kind == k; }
    bool atEnd() const { return check(TokenKind::EndOfFile); }
    const Token& advance() { if (!atEnd()) pos_++; return previous(); }
    bool match(TokenKind k) { if (check(k)) { advance(); return true; } return
    const Token& expect(TokenKind k, const std::string& msg);
    void synchronize(); // panic-mode recovery

    FunctionDecl parseFunction();
    StmtPtr parseStatement();
    StmtPtr parseVarDecl();
    std::unique_ptr<BlockStmt> parseBlock();
    StmtPtr parseIf();
    StmtPtr parseWhile();
    StmtPtr parseReturn();
    StmtPtr parseExprStmt();

    // Precedence-climbing expression parsers, one per tier, from the grammar a
    ExprPtr parseExpr();        // = Assignment
    ExprPtr parseAssignment();
    ExprPtr parseEquality();
    ExprPtr parseComparison();
    ExprPtr parseTerm();
    ExprPtr parseFactor();
    ExprPtr parseUnary();
    ExprPtr parsePrimary();
};
```

```
} // namespace lumen
```

cpp

```cpp
#include "parser.hpp"
#include <stdexcept>

namespace lumen {

const Token& Parser::expect(TokenKind k, const std::string& msg) {
    if (check(k)) return advance();
    throw std::runtime_error(
        "parse error at line " + std::to_string(peek().line) + ": " + msg);
}

// After a syntax error, skip tokens until we're at a likely statement
// boundary (';' or '}') so subsequent statements can still be checked —
// otherwise one typo produces a cascade of bogus follow-on errors.
void Parser::synchronize() {
    advance();
    while (!atEnd()) {
        if (previous().kind == TokenKind::Semicolon) return;
        if (peek().kind == TokenKind::RBrace) return;
        advance();
    }
}

Program Parser::parseProgram() {
    Program program;
    while (!atEnd()) {
        program.functions.push_back(parseFunction());
    }
    return program;
}

FunctionDecl Parser::parseFunction() {
    expect(TokenKind::KwInt, "expected return type 'int'");
    Token name = expect(TokenKind::Identifier, "expected function name");
    expect(TokenKind::LParen, "expected '('");
    expect(TokenKind::RParen, "expected ')' (no params yet in Lumen v0)");
    auto body = parseBlock();
    return FunctionDecl{std::string(name.lexeme), std::move(body)};
}

std::unique_ptr<BlockStmt> Parser::parseBlock() {
    expect(TokenKind::LBrace, "expected '{'");
    auto block = std::make_unique<BlockStmt>();
    while (!check(TokenKind::RBrace) && !atEnd()) {
        try {
            block->statements.push_back(parseStatement());
```

```
        } catch (const std::runtime_error& e) {
            // report and recover instead of aborting the whole parse
            fprintf(stderr, "%s\n", e.what());
            synchronize();
        }
    }
    expect(TokenKind::RBrace, "expected '}'");
    return block;
}

StmtPtr Parser::parseStatement() {
    if (check(TokenKind::KwInt))    return parseVarDecl();
    if (check(TokenKind::KwIf))     return parseIf();
    if (check(TokenKind::KwWhile))  return parseWhile();
    if (check(TokenKind::KwReturn)) return parseReturn();
    if (check(TokenKind::LBrace))   return parseBlock();
    return parseExprStmt();
}

StmtPtr Parser::parseVarDecl() {
    advance(); // 'int'
    Token name = expect(TokenKind::Identifier, "expected variable name");
    auto decl = std::make_unique<VarDeclStmt>();
    decl->name = std::string(name.lexeme);
    if (match(TokenKind::Equals)) decl->init = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after variable declaration");
    return decl;
}

StmtPtr Parser::parseIf() {
    advance(); // 'if'
    expect(TokenKind::LParen, "expected '(' after 'if'");
    auto stmt = std::make_unique<IfStmt>();
    stmt->cond = parseExpr();
    expect(TokenKind::RParen, "expected ')'");
    stmt->thenBranch = parseStatement();
    if (match(TokenKind::KwElse)) stmt->elseBranch = parseStatement();
    return stmt;
}

StmtPtr Parser::parseWhile() {
    advance(); // 'while'
    expect(TokenKind::LParen, "expected '(' after 'while'");
    auto stmt = std::make_unique<WhileStmt>();
    stmt->cond = parseExpr();
    expect(TokenKind::RParen, "expected ')'");
    stmt->body = parseStatement();
```

```
    return stmt;
}

StmtPtr Parser::parseReturn() {
    advance(); // 'return'
    auto stmt = std::make_unique<ReturnStmt>();
    if (!check(TokenKind::Semicolon)) stmt->value = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after return");
    return stmt;
}

StmtPtr Parser::parseExprStmt() {
    auto expr = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after expression");
    auto stmt = std::make_unique<ExprStmt>();
    stmt->expr = std::move(expr);
    return stmt;
}

ExprPtr Parser::parseExpr() { return parseAssignment(); }

ExprPtr Parser::parseAssignment() {
    ExprPtr expr = parseEquality();
    if (match(TokenKind::Equals)) {
        // Only a bare identifier is a legal assignment target in Lumen v0.
        if (auto* var = dynamic_cast<VarExpr*>(expr.get())) {
            auto assign = std::make_unique<AssignExpr>();
            assign->name = var->name;
            assign->value = parseAssignment(); // right-associative
            return assign;
        }
        throw std::runtime_error("invalid assignment target");
    }
    return expr;
}

// Shared shape for every left-associative binary tier: parse one operand
// at the next tier down, then loop consuming same-precedence operators.
ExprPtr Parser::parseEquality() {
    ExprPtr expr = parseComparison();
    while (check(TokenKind::EqualsEquals)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseComparison();
        expr = std::move(bin);
    }
    return expr;
```

```
}

ExprPtr Parser::parseComparison() {
    ExprPtr expr = parseTerm();
    while (check(TokenKind::Less) || check(TokenKind::LessEqual) ||
           check(TokenKind::Greater) || check(TokenKind::GreaterEqual)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseTerm();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseTerm() {
    ExprPtr expr = parseFactor();
    while (check(TokenKind::Plus) || check(TokenKind::Minus)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseFactor();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseFactor() {
    ExprPtr expr = parseUnary();
    while (check(TokenKind::Star) || check(TokenKind::Slash)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseUnary();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseUnary() {
    if (check(TokenKind::Minus)) {
        TokenKind op = advance().kind;
        auto un = std::make_unique<UnaryExpr>();
        un->op = op; un->operand = parseUnary();
        return un;
    }
    return parsePrimary();
}

ExprPtr Parser::parsePrimary() {
```

```
    if (check(TokenKind::Number)) {
        auto tok = advance();
        auto n = std::make_unique<NumberExpr>();
        n->value = std::stoi(std::string(tok.lexeme));
        return n;
    }
    if (check(TokenKind::Identifier)) {
        auto tok = advance();
        auto v = std::make_unique<VarExpr>();
        v->name = std::string(tok.lexeme);
        return v;
    }
    if (match(TokenKind::LParen)) {
        auto expr = parseExpr();
        expect(TokenKind::RParen, "expected ')'");
        return expr;
    }
    throw std::runtime_error(
        "parse error at line " + std::to_string(peek().line) + ": expected expr
}

} // namespace lumen
```

Every tier from `parseEquality` down to `parseFactor` has the identical shape — `operand = lower(); while (op at this precedence) { operand = Binary(op, operand, lower()) }` — which *is* precedence climbing. This directly answers exercise-style question: left recursion in the grammar becomes a `while` loop in code, at every tier uniformly.

2.6 Exercises

1. Trace by hand how `2 + 3 * 4` parses — confirm `*` binds tighter (it goes through one more precedence tier before `+` ever sees it).
2. Add `&&`/`||` as new precedence tiers between `Assignment` and `Equality`, with correct relative precedence to C-family languages.
3. **Debugging exercise:** the current `synchronize()` only stops at `;` or `}`. Construct a Lumen snippet where this still produces a cascading garbage error, and fix it.
4. **Interview-style:** why is right-associativity for `Assignment` (`a = b = c` means `a = (b = c)`) implemented via recursion (`parseAssignment` calling itself) while left-associativity for `+` is implemented via a loop? What would go wrong if you used a loop for assignment?

2.7 Real compiler comparison

Clang's `Parser::ParseExpression` and friends are structurally identical to what's above — one method per precedence tier, explicit loops for left-associative operators — scaled up with a full operator-precedence table for C++'s much larger operator set instead of hand-tiered methods. rustc's parser is also hand-written recursive descent. GCC's C frontend

historically used a Bison (LALR) grammar but migrated much of its expression parsing to hand-written code for the same diagnostic-quality reasons discussed in Phase 1.

Phase 3 — Semantic Analysis

Semantic analysis walks the (now syntactically valid) AST and asks: does this make sense? Two core mechanisms:

- **Symbol tables & scope:** a stack of hash maps (one per lexical scope), pushed on block entry, popped on exit. Name lookup walks the stack from innermost to outermost — this *is* lexical scoping, implemented directly.
- **Type checking:** each AST node gets an inferred/declared type; expressions are checked bottom-up (children's types determine what operators are legal). Lumen starts with a single type (`int`) plus implicit booleans-as-ints for conditions (like C) — full type inference (Hindley-Milner, as in ML/Haskell/Rust) is a Phase-9 extension.

Design trade-off worth internalizing: do semantic checks *mutate* the AST in place (annotate nodes with resolved symbols) or build a separate typed representation? Clang mutates/annotates in place; some functional-language compilers build a fresh typed tree. We annotate in place for Lumen — simpler, and what most C-family compilers do.

3.2 Symbol table

`src/sema/symbol_table.hpp`:

```cpp
```

```cpp
#pragma once
#include <string>
#include <unordered_map>
#include <vector>
#include <stdexcept>

namespace lumen {

struct Symbol {
    std::string name;
    // Only one type exists today, but this field is where a real type
    // system plugs in later without touching the scope-walking logic.
    std::string type = "int";
};

// One hash map per lexical scope, stacked. Push on block entry, pop on
// exit — this literally *is* lexical scoping, not a simulation of it.
class SymbolTable {
public:
    void pushScope() { scopes_.emplace_back(); }
    void popScope()  { scopes_.pop_back(); }

    // Declares in the *innermost* scope; rejects redeclaration in that
    // same scope (shadowing an outer scope is fine, matching C semantics).
    void declare(const Symbol& sym) {
        auto& innermost = scopes_.back();
        if (innermost.count(sym.name)) {
            throw std::runtime_error("redeclaration of '" + sym.name + "' in sa
        }
        innermost[sym.name] = sym;
    }

    // Walks from innermost to outermost — the direct implementation of
    // "closest enclosing declaration wins."
    const Symbol* resolve(const std::string& name) const {
        for (auto it = scopes_.rbegin(); it != scopes_.rend(); ++it) {
            auto found = it->find(name);
            if (found != it->end()) return &found->second;
        }
        return nullptr;
    }

private:
    std::vector<std::unordered_map<std::string, Symbol>> scopes_;
};
```

```
} // namespace lumen
```

3.3 The semantic analyzer pass

`src/sema/sema.hpp` / `.cpp` — a tree walk that (a) resolves every `VarExpr`/`AssignExpr` against the symbol table and (b) reports use-before-declaration and redeclaration errors. (Type-checking arithmetic is trivial while there's only one type — the structure below is exactly where a second type would plug in, in `checkBinary`.)

cpp

```cpp
#pragma once
#include "../ast/ast.hpp"
#include "symbol_table.hpp"

namespace lumen {

class Sema {
public:
    // Returns true if no errors were found.
    bool check(Program& program);

private:
    SymbolTable symbols_;
    bool ok_ = true;

    void checkFunction(FunctionDecl& fn);
    void checkStmt(Stmt* stmt);
    void checkExpr(Expr* expr);

    void error(int line, const std::string& msg);
};

} // namespace lumen
```

cpp

```
#include "sema.hpp"
#include <cstdio>

namespace lumen {

void Sema::error(int line, const std::string& msg) {
    fprintf(stderr, "semantic error at line %d: %s\n", line, msg.c_str());
    ok_ = false;
}

bool Sema::check(Program& program) {
    for (auto& fn : program.functions) checkFunction(fn);
    return ok_;
}

void Sema::checkFunction(FunctionDecl& fn) {
    symbols_.pushScope();
    for (auto& stmt : fn.body->statements) checkStmt(stmt.get());
    symbols_.popScope();
}

void Sema::checkStmt(Stmt* stmt) {
    if (auto* decl = dynamic_cast<VarDeclStmt*>(stmt)) {
        if (decl->init) checkExpr(decl->init.get());
        // Declare *after* checking the initializer, so `int x = x;` is
        // correctly rejected as use-before-declaration rather than
        // silently resolving to itself.
        try {
            symbols_.declare(Symbol{decl->name});
        } catch (const std::exception& e) {
            error(decl->line, e.what());
        }
    } else if (auto* block = dynamic_cast<BlockStmt*>(stmt)) {
        symbols_.pushScope();
        for (auto& s : block->statements) checkStmt(s.get());
        symbols_.popScope();
    } else if (auto* ifs = dynamic_cast<IfStmt*>(stmt)) {
        checkExpr(ifs->cond.get());
        checkStmt(ifs->thenBranch.get());
        if (ifs->elseBranch) checkStmt(ifs->elseBranch.get());
    } else if (auto* whiles = dynamic_cast<WhileStmt*>(stmt)) {
        checkExpr(whiles->cond.get());
        checkStmt(whiles->body.get());
    } else if (auto* ret = dynamic_cast<ReturnStmt*>(stmt)) {
        if (ret->value) checkExpr(ret->value.get());
    } else if (auto* es = dynamic_cast<ExprStmt*>(stmt)) {
```

```
        checkExpr(es->expr.get());
    }
}

void Sema::checkExpr(Expr* expr) {
    if (auto* var = dynamic_cast<VarExpr*>(expr)) {
        if (!symbols_.resolve(var->name)) {
            error(var->line, "use of undeclared identifier '" + var->name + "'");
        }
    } else if (auto* assign = dynamic_cast<AssignExpr*>(expr)) {
        if (!symbols_.resolve(assign->name)) {
            error(assign->line, "assignment to undeclared identifier '" + assig
        }
        checkExpr(assign->value.get());
    } else if (auto* bin = dynamic_cast<BinaryExpr*>(expr)) {
        checkExpr(bin->lhs.get());
        checkExpr(bin->rhs.get());
        // checkBinary(bin) is where a real type system would verify
        // operand types are compatible with `bin->op` and record the
        // result type on the node — the plug point mentioned above.
    } else if (auto* un = dynamic_cast<UnaryExpr*>(expr)) {
        checkExpr(un->operand.get());
    }
    // NumberExpr: nothing to check, it's always valid.
}

} // namespace lumen
```

Notice the `dynamic_cast` chain — this is the cost of *not* having a visitor pattern yet. It's honest, readable, and O(1) casts are cheap at compiler scale, but it's also exactly the pain point that motivates introducing a visitor once we have three or four passes walking the same tree shape (sema, IR gen, and later an AST printer for debugging) — each new pass repeats this same `dynamic_cast` boilerplate. **Exercise 3.4 below has you fix this properly.**

3.4 Exercises

1. Extend `Symbol` and the grammar with a `bool` type; make `if`/`while` conditions require a value (int-as-truthy is fine, matching C, but *record* that decision rather than leaving it implicit).
2. **Debugging exercise:** `int x = x;` — trace through `checkStmt`/`checkExpr` by hand and confirm it's rejected. Now trace `{ int x = 1; { int x = x; } }` — is the inner `x` correctly resolved to the *outer* `x` on its right-hand side? Why or why not, given declare-after-check-initializer ordering?
3. **Refactor exercise (real one, not hypothetical):** replace the `dynamic_cast` chains in `checkStmt`/`checkExpr` with a proper visitor pattern (virtual `accept(Visitor&)` on

each AST node). This is the point in a real project where that abstraction pays for itself — you'll reuse the same `Visitor` base for IR generation in Phase 4.

4. **Interview-style:** why must `VarDeclStmt`'s initializer be checked *before* the name is declared (rather than declaring first, then checking the initializer)? Give a concrete Lumen snippet where the wrong order silently accepts invalid code.

3.5 Real compiler comparison

Clang's `Sema` class is the single largest component in the codebase by a wide margin — because real type systems (overload resolution, implicit conversions, templates) turn "does this make sense" into the hardest part of the whole compiler, dwarfing lexing/parsing. rustc's borrow checker is *also* a semantic-analysis-phase concept, just a far more elaborate one (tracking ownership/lifetimes, not just types) — the symbol-table-and-scope machinery above is the same load-bearing structure underneath it, scaled up enormously.

Phase 4 — Intermediate Representation

Why not go straight from AST to assembly? Because AST is language-shaped and assembly is machine-shaped — coupling them directly means every new source feature needs matching logic in every backend. **IR decouples the two.**

- **Three-address code (TAC):** each instruction has at most one operator and three operands (`t1 = t2 + t3`) — matches roughly what most CPU instructions can do, making later code generation nearly mechanical.
- **SSA (Static Single Assignment):** every variable is assigned exactly once; a `phi` node merges values from different control-flow paths. This is what **LLVM IR** is built on, because SSA makes data-flow analysis (needed for optimization) dramatically simpler — "where was this value defined" becomes a single unambiguous answer instead of a search.
- **Control-flow graphs (CFG):** basic blocks (straight-line instruction sequences) as nodes, jumps/branches as edges. Nearly every optimization is phrased as "a computation over the CFG."

Lumen lowers AST → TAC directly (simplest to generate from a tree walk); converting TAC → SSA is Exercise 4.4 below rather than built into the main pipeline, since Lumen's optimizer (Phase 5) doesn't strictly require SSA for the passes we implement — but the conversion is worth doing once by hand to *see* why LLVM is built the way it is.

4.2 IR data structures

`src/ir/ir.hpp`:

```cpp
```

```cpp
#pragma once
#include <string>
#include <vector>
#include <variant>
#include <memory>

namespace lumen::ir {

// An operand is either a compile-time constant or a virtual register
// name ("t1", "t2", ...) — unlimited, unlike real machine registers.
// Keeping this as a tagged union (not a class hierarchy) keeps every
// later pass (optimizer, codegen) a simple switch instead of virtual
// dispatch, which matters because these are visited *very* often.
struct Operand {
    enum class Kind { Const, Temp, Var } kind;
    int constValue = 0;
    std::string name; // temp or variable name
};

enum class Op {
    Add, Sub, Mul, Div,
    Lt, Le, Gt, Ge, Eq, Ne,
    Neg,
    Copy,        // dst = src
    LoadConst,   // dst = constant
    Jump,        // unconditional jump to a block
    CondJump,    // if operand: jump to block A else block B
    Return,
};

struct Instr {
    Op op;
    Operand dst;                // result temp (unused for Jump)
    std::vector<Operand> args;  // 0-2 source operands
    std::string targetBlock;    // for Jump
    std::string thenBlock, elseBlock; // for CondJump
};

// A basic block: a straight-line instruction sequence with exactly one
// entry (its label) and one exit (the last instruction, always a
// jump/condjump/return) — the CFG's node type.
struct BasicBlock {
    std::string label;
    std::vector<Instr> instrs;
};
```

```
struct Function {
    std::string name;
    std::vector<BasicBlock> blocks;
};

} // namespace lumen::ir
```

4.3 AST → IR lowering

The generator walks the AST once, emitting instructions into "the current block," creating new blocks whenever control flow branches (`if`/`while`). This is the single trickiest part of the whole compiler conceptually — not because any one step is hard, but because you're simultaneously (a) evaluating expressions into temporaries and (b) restructuring tree-shaped control flow into a flat graph of blocks.

`src/ir/irgen.hpp` / `.cpp`:

cpp

```
#pragma once
#include "../ast/ast.hpp"
#include "ir.hpp"
#include <unordered_map>

namespace lumen {

class IRGen {
public:
    ir::Function generate(FunctionDecl& fn);

private:
    ir::Function* current_ = nullptr;
    int tempCounter_ = 0;
    int labelCounter_ = 0;

    ir::BasicBlock* currentBlock_ = nullptr;

    std::string newTemp()  { return "t" + std::to_string(tempCounter_++); }
    std::string newLabel(const std::string& base) {
        return base + std::to_string(labelCounter_++);
    }
    ir::BasicBlock& newBlock(const std::string& label);
    void emit(ir::Instr instr) { currentBlock_->instrs.push_back(std::move(inst

    // Returns the operand holding the expression's result (a temp, a
    // variable reference, or a constant — no need to force everything
    // through a temp if it's already a leaf value).
    ir::Operand genExpr(Expr* expr);
    void genStmt(Stmt* stmt);
};

} // namespace lumen
```

cpp

```cpp
#include "irgen.hpp"

namespace lumen {

ir::BasicBlock& IRGen::newBlock(const std::string& label) {
    current_->blocks.push_back(ir::BasicBlock{label, {}});
    currentBlock_ = &current_->blocks.back();
    return current_->blocks.back();
}

ir::Function IRGen::generate(FunctionDecl& fn) {
    ir::Function irFn;
    irFn.name = fn.name;
    current_ = &irFn;
    newBlock("entry");
    for (auto& stmt : fn.body->statements) genStmt(stmt.get());
    current_ = nullptr;
    return irFn;
}

static ir::Op binOpFor(TokenKind k) {
    using ir::Op;
    switch (k) {
        case TokenKind::Plus:          return Op::Add;
        case TokenKind::Minus:         return Op::Sub;
        case TokenKind::Star:          return Op::Mul;
        case TokenKind::Slash:         return Op::Div;
        case TokenKind::Less:          return Op::Lt;
        case TokenKind::LessEqual:     return Op::Le;
        case TokenKind::Greater:       return Op::Gt;
        case TokenKind::GreaterEqual:  return Op::Ge;
        case TokenKind::EqualsEquals:  return Op::Eq;
        default: throw std::runtime_error("unsupported binary op in IR gen");
    }
}

ir::Operand IRGen::genExpr(Expr* expr) {
    using ir::Operand;

    if (auto* n = dynamic_cast<NumberExpr*>(expr)) {
        return Operand{Operand::Kind::Const, n->value, ""};
    }
    if (auto* v = dynamic_cast<VarExpr*>(expr)) {
        return Operand{Operand::Kind::Var, 0, v->name};
    }
    if (auto* a = dynamic_cast<AssignExpr*>(expr)) {
```

```
        Operand value = genExpr(a->value.get());
        emit(ir::Instr{ir::Op::Copy, Operand{Operand::Kind::Var, 0, a->name}, {
        return Operand{Operand::Kind::Var, 0, a->name};
    }
    if (auto* u = dynamic_cast<UnaryExpr*>(expr)) {
        Operand operand = genExpr(u->operand.get());
        std::string t = newTemp();
        emit(ir::Instr{ir::Op::Neg, Operand{Operand::Kind::Temp, 0, t}, {operan
        return Operand{Operand::Kind::Temp, 0, t};
    }
    if (auto* b = dynamic_cast<BinaryExpr*>(expr)) {
        Operand lhs = genExpr(b->lhs.get());
        Operand rhs = genExpr(b->rhs.get());
        std::string t = newTemp();
        emit(ir::Instr{binOpFor(b->op), Operand{Operand::Kind::Temp, 0, t}, {lh
        return Operand{Operand::Kind::Temp, 0, t};
    }
    throw std::runtime_error("unhandled expression kind in IR gen");
}

void IRGen::genStmt(Stmt* stmt) {
    using ir::Operand;

    if (auto* decl = dynamic_cast<VarDeclStmt*>(stmt)) {
        if (decl->init) {
            Operand value = genExpr(decl->init.get());
            emit(ir::Instr{ir::Op::Copy, Operand{Operand::Kind::Var, 0, decl->n
        }
    } else if (auto* es = dynamic_cast<ExprStmt*>(stmt)) {
        genExpr(es->expr.get());
    } else if (auto* block = dynamic_cast<BlockStmt*>(stmt)) {
        for (auto& s : block->statements) genStmt(s.get());
    } else if (auto* ifs = dynamic_cast<IfStmt*>(stmt)) {
        // This is the heart of tree-to-graph lowering: one AST node
        // becomes three (or two) basic blocks wired together by jumps.
        Operand cond = genExpr(ifs->cond.get());
        std::string thenLabel = newLabel("if_then");
        std::string elseLabel = newLabel("if_else");
        std::string endLabel  = newLabel("if_end");
        bool hasElse = ifs->elseBranch != nullptr;

        emit(ir::Instr{ir::Op::CondJump, {}, {cond}, "", thenLabel,
                       hasElse ? elseLabel : endLabel});

        newBlock(thenLabel);
        genStmt(ifs->thenBranch.get());
        emit(ir::Instr{ir::Op::Jump, {}, {}, endLabel});
```

```
        if (hasElse) {
            newBlock(elseLabel);
            genStmt(ifs->elseBranch.get());
            emit(ir::Instr{ir::Op::Jump, {}, {}, endLabel});
        }

        newBlock(endLabel);
    } else if (auto* whiles = dynamic_cast<WhileStmt*>(stmt)) {
        std::string condLabel = newLabel("while_cond");
        std::string bodyLabel = newLabel("while_body");
        std::string endLabel  = newLabel("while_end");

        emit(ir::Instr{ir::Op::Jump, {}, {}, condLabel});

        newBlock(condLabel);
        Operand cond = genExpr(whiles->cond.get());
        emit(ir::Instr{ir::Op::CondJump, {}, {cond}, "", bodyLabel, endLabel});

        newBlock(bodyLabel);
        genStmt(whiles->body.get());
        emit(ir::Instr{ir::Op::Jump, {}, {}, condLabel}); // back edge — the lo

        newBlock(endLabel);
    } else if (auto* ret = dynamic_cast<ReturnStmt*>(stmt)) {
        Operand value = ret->value
            ? genExpr(ret->value.get())
            : Operand{Operand::Kind::Const, 0, ""};
        emit(ir::Instr{ir::Op::Return, {}, {value}});
    }
}

} // namespace lumen
```

The `while` loop's *back edge* — the final `Jump` to `condLabel` — is what makes this a *cyclic* graph, and is exactly the structure every later data-flow analysis (Phase 5) has to handle correctly (a naive single forward pass over blocks isn't enough once cycles exist, which is why Kildall's algorithm iterates to a fixed point rather than doing one pass).

4.4 Exercises

1. Trace by hand the IR generated for:

```
int x = 0;
while (x < 5) { x = x + 1; }
return x;
```

Draw the resulting CFG (blocks + edges) on paper before checking it against the code. 2. Add `&&`/`||` — note these need **short-circuit** control flow (don't evaluate the right side if the left already determines the result), so they lower to *branches*, not a single `Op::And` instruction. This is a good test of whether you've internalized the `if`-lowering pattern above. 3. **Conversion exercise:** hand-convert the `while` example's TAC into SSA — introduce a `phi` node at `while_cond` merging `x`'s value from `entry` and from `while_body`. This single exercise is the fastest route to understanding why LLVM IR looks the way it does. 4. **Interview-style:** why does `genExpr` return an `Operand` (which might already be a variable or constant) instead of *always* emitting a `Copy` into a fresh temp? What would the cost be, and which later pass would have to clean it up if we did?

4.5 Real compiler comparison

LLVM IR is SSA-form from the moment a frontend (Clang, rustc) emits it — frontends are expected to produce SSA directly (often via a simple "stack-allocate everything, then run `mem2reg`" trick, which mechanizes exactly the TAC→SSA conversion from Exercise 4.4.3) rather than lowering to non-SSA TAC first. GCC's GIMPLE is conceptually TAC-like before its own SSA pass runs on top. Our two-step approach (TAC first, SSA as a deliberate later exercise) mirrors GCC's historical path more than LLVM's frontend contract, and is the gentler on-ramp pedagogically.

Phase 5 — Optimization

Optimizations are **CFG/IR rewrites that preserve meaning while improving some cost metric** (speed, size). Categories:

- **Local** (within one basic block): constant folding (`2+3` → `5` at compile time), common subexpression elimination, local dead-code elimination.
- **Global** (across the CFG): requires data-flow analysis — e.g. **liveness analysis** (is this value still needed later?) drives both dead-code elimination and register allocation; **reaching definitions** drives constant propagation across blocks.
- Data-flow analyses are all instances of one template: iterate over the CFG applying a transfer function per block until a fixed point is reached (Kildall's algorithm). This single pattern underlies liveness, reaching definitions, available expressions, and more.

5.2 Constant folding (local optimization)

The simplest useful optimization: if both operands of a binary IR instruction are constants, compute the result at compile time and replace the instruction with a `LoadConst`.

`src/ir/optimize.hpp` / `.cpp`:

```cpp
```

```cpp
#pragma once
#include "ir.hpp"

namespace lumen::opt {
// Returns true if it changed anything (callers loop until it returns
// false — folding can expose new folding opportunities, e.g.
// `t1 = 2 + 3; t2 = t1 * 4;` needs two passes to fully collapse).
bool constantFold(ir::Function& fn);
}
```

cpp

```cpp
#include "optimize.hpp"

namespace lumen::opt {

static bool tryFold(ir::Instr& instr) {
    using ir::Op;
    using ir::Operand;
    if (instr.args.size() != 2) return false;
    auto& a = instr.args[0];
    auto& b = instr.args[1];
    if (a.kind != Operand::Kind::Const || b.kind != Operand::Kind::Const) retur

    int result;
    switch (instr.op) {
        case Op::Add: result = a.constValue + b.constValue; break;
        case Op::Sub: result = a.constValue - b.constValue; break;
        case Op::Mul: result = a.constValue * b.constValue; break;
        case Op::Div:
            if (b.constValue == 0) return false; // don't fold div-by-zero away
            result = a.constValue / b.constValue;
            break;
        case Op::Lt: result = a.constValue <  b.constValue; break;
        case Op::Le: result = a.constValue <= b.constValue; break;
        case Op::Gt: result = a.constValue >  b.constValue; break;
        case Op::Ge: result = a.constValue >= b.constValue; break;
        case Op::Eq: result = a.constValue == b.constValue; break;
        default: return false;
    }
    instr.op = Op::LoadConst;
    instr.args = {Operand{Operand::Kind::Const, result, ""}};
    return true;
}

bool constantFold(ir::Function& fn) {
    bool changed = false;
    for (auto& block : fn.blocks)
        for (auto& instr : block.instrs)
            changed |= tryFold(instr);
    return changed;
}

} // namespace lumen::opt
```

Driving it to a fixed point in `main`: `while (opt::constantFold(fn)) {}` — this is the smallest possible instance of "iterate to a fixed point," the same principle Phase 5.3's liveness

analysis uses at CFG scale instead of instruction scale.

5.3 Liveness analysis (global, the Kildall's-algorithm template)

A variable/temp is **live** at a point if its current value might be used later before being overwritten. Liveness is computed **backward** over the CFG (unlike most analyses, which run forward) because "will this be used later" is naturally a question about the future relative to each point:

```
live_out[B] = ∪ live_in[S] for each successor S of B
live_in[B]  = use[B] ∪ (live_out[B] − def[B])
```

where `use[B]` = temps/vars read before being written in `B`, `def[B]` = vars written in `B`. Iterate over all blocks, recomputing `live_in`/`live_out`, until nothing changes — the fixed point. This directly feeds **dead code elimination** (an instruction whose result is never live can be deleted) and **register allocation** (Phase 6 — two values needing a register at the same time is exactly "both live simultaneously").

cpp

```cpp
#pragma once
#include "ir.hpp"
#include <unordered_set>
#include <unordered_map>
#include <string>

namespace lumen::opt {

struct LivenessResult {
    std::unordered_map<std::string, std::unordered_set<std::string>> liveIn, li
};

LivenessResult computeLiveness(const ir::Function& fn);

}
```

cpp

```
#include "liveness.hpp"

namespace lumen::opt {

using Set = std::unordered_set<std::string>;

// Which names an instruction reads (`use`) and writes (`def`).
static void useDef(const ir::Instr& instr, Set& use, Set& def) {
    for (auto& arg : instr.args)
        if (arg.kind != ir::Operand::Kind::Const) use.insert(arg.name);
    if (instr.op == ir::Op::Copy || instr.dst.kind != ir::Operand::Kind::Const)
        if (!instr.dst.name.empty()) def.insert(instr.dst.name);
}

// Build a simple successor map by scanning each block's terminator —
// a real implementation would build this once as part of the CFG
// structure itself rather than re-deriving it here.
static std::unordered_map<std::string, std::vector<std::string>>
successors(const ir::Function& fn) {
    std::unordered_map<std::string, std::vector<std::string>> succ;
    for (auto& block : fn.blocks) {
        if (block.instrs.empty()) continue;
        auto& last = block.instrs.back();
        if (last.op == ir::Op::Jump) succ[block.label] = {last.targetBlock};
        else if (last.op == ir::Op::CondJump) succ[block.label] = {last.thenBlo
        // Return: no successors.
    }
    return succ;
}

LivenessResult computeLiveness(const ir::Function& fn) {
    LivenessResult result;
    auto succ = successors(fn);
    bool changed = true;
    while (changed) {
        changed = false;
        // Iterate blocks in reverse order — liveness is a backward
        // analysis, so processing successors-before-predecessors
        // converges faster (though correctness doesn't depend on order,
        // only the fixed point does — order is a speed optimization).
        for (auto it = fn.blocks.rbegin(); it != fn.blocks.rend(); ++it) {
            auto& block = *it;
            Set newOut;
            for (auto& s : succ[block.label])
                newOut.insert(result.liveIn[s].begin(), result.liveIn[s].end())
```

```
            Set use, def;
            for (auto& instr : block.instrs) useDef(instr, use, def);

            Set newIn = use;
            for (auto& v : newOut) if (!def.count(v)) newIn.insert(v);

            if (newIn != result.liveIn[block.label] || newOut != result.liveOut
                result.liveIn[block.label] = newIn;
                result.liveOut[block.label] = newOut;
                changed = true;
            }
        }
    }
    return result;
}

} // namespace lumen::opt
```

5.4 Exercises

1. Run constant folding on `int x = 2 + 3 * 4;` — confirm it takes two iterations (fold the `*` first, then the `+`) and verify your `while (constantFold(fn))` driver handles that.
2. Implement **dead code elimination** using the liveness result: delete any instruction whose `dst` is not in that program point's live set (careful: instructions with side effects — none exist in Lumen yet, but `Return`/`CondJump` must never be deleted regardless of liveness).
3. **Debugging exercise:** construct a Lumen snippet with a loop where a naive *forward-only, single-pass* liveness computation gives a wrong answer, and explain why the fixed-point iteration is what fixes it.
4. **Interview-style:** why is liveness computed backward while reaching-definitions (a similar-looking analysis, used for constant propagation across blocks) is computed forward? What property of each question ("will this be used" vs "what was last assigned here") determines the direction?

5.5 Real compiler comparison

LLVM's optimizer is a pipeline of dozens of such passes (`-O2` runs ~150), each independently testable and mostly following this exact use/def + fixed-point shape, orchestrated by a pass manager that also handles pass ordering and dependency invalidation (a pass that changes the CFG invalidates any previously computed liveness/dominance results, forcing a recompute) — a bookkeeping problem we're sidestepping in Lumen by just re-running everything from scratch each time, which is correct but far slower, exactly the kind of trade-off production compilers can't afford at scale.

Phase 6 — Code Generation

- **Instruction selection:** mapping IR ops to real CPU instructions — often via tree pattern matching (e.g. `t1 = a * b + c` might become a single `imul`+`add`, or on some architectures a fused multiply-add).
- **Register allocation:** IR uses unlimited virtual registers; real CPUs have ~16 general-purpose ones. **Linear scan** (fast, used in JITs like V8) vs **graph coloring** (slower, better code, used in GCC/LLVM's `-O2`+) is the central trade-off — build time vs runtime quality.
- **Calling conventions & stack frames:** how arguments/return values move between caller and callee (registers vs stack, per-ABI rules), and how each function's local storage is laid out on the stack (prologue allocates, epilogue restores).

6.2 Lumen's codegen strategy: everything on the stack

Real register allocation (linear scan / graph coloring) is a substantial project on its own — Lumen's first working codegen deliberately **skips it** by giving every variable and temp its own stack slot (the "naive" strategy every teaching compiler, and even early production compilers historically, starts with). This produces correct but unoptimized assembly; you add real register allocation as a Phase-9-style extension once this baseline works end-to-end. This trade-off — correctness and simplicity first, performance later, as a clearly separated pass — mirrors how real compiler projects are actually staged.

`src/codegen/codegen.hpp` / `.cpp` (targeting x86-64, System V ABI, Linux/macOS):

```cpp
```

```cpp
#pragma once
#include "../ir/ir.hpp"
#include <sstream>
#include <unordered_map>

namespace lumen {

class CodeGen {
public:
    std::string generate(const ir::Function& fn);

private:
    std::ostringstream out_;
    std::unordered_map<std::string, int> slots_; // name -> stack offset
    int nextOffset_ = 0;

    int slotFor(const std::string& name);
    std::string operandRef(const ir::Operand& op); // e.g. "-8(%rbp)" or "$5"
    void emitInstr(const ir::Instr& instr);
};

}
```

cpp

```cpp
#include "codegen.hpp"

namespace lumen {

int CodeGen::slotFor(const std::string& name) {
    auto it = slots_.find(name);
    if (it != slots_.end()) return it->second;
    nextOffset_ += 8; // one 8-byte slot per value, no packing/optimization
    slots_[name] = nextOffset_;
    return nextOffset_;
}

std::string CodeGen::operandRef(const ir::Operand& op) {
    if (op.kind == ir::Operand::Kind::Const) {
        return "$" + std::to_string(op.constValue);
    }
    return "-" + std::to_string(slotFor(op.name)) + "(%rbp)";
}

void CodeGen::emitInstr(const ir::Instr& instr) {
    using Op = ir::Op;
    // Stack-machine style: load operands into %rax/%rcx, compute, store
    // the result back to the destination's stack slot. Wasteful (every
    // value round-trips through memory) but simple and obviously
    // correct — exactly the point of a first codegen pass.
    switch (instr.op) {
        case Op::LoadConst:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        case Op::Copy:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        case Op::Add: case Op::Sub: case Op::Mul: {
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  movq " << operandRef(instr.args[1]) << ", %rcx\n";
            if (instr.op == Op::Add) out_ << "  addq %rcx, %rax\n";
            if (instr.op == Op::Sub) out_ << "  subq %rcx, %rax\n";
            if (instr.op == Op::Mul) out_ << "  imulq %rcx, %rax\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        }
        case Op::Div:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  cqto\n"; // sign-extend %rax into %rdx:%rax for idiv
```

```
            out_ << "  movq " << operandRef(instr.args[1]) << ", %rcx\n";
            out_ << "  idivq %rcx\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        case Op::Lt: case Op::Le: case Op::Gt: case Op::Ge: case Op::Eq: {
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  movq " << operandRef(instr.args[1]) << ", %rcx\n";
            out_ << "  cmpq %rcx, %rax\n";
            std::string set = instr.op == Op::Lt ? "setl" : instr.op == Op::Le
                            : instr.op == Op::Gt ? "setg" : instr.op == Op::Ge
            out_ << "  " << set << " %al\n";
            out_ << "  movzbq %al, %rax\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        }
        case Op::Neg:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  negq %rax\n";
            out_ << "  movq %rax, " << operandRef(instr.dst) << "\n";
            break;
        case Op::Jump:
            out_ << "  jmp ." << instr.targetBlock << "\n";
            break;
        case Op::CondJump:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  cmpq $0, %rax\n";
            out_ << "  je ." << instr.elseBlock << "\n";
            out_ << "  jmp ." << instr.thenBlock << "\n";
            break;
        case Op::Return:
            out_ << "  movq " << operandRef(instr.args[0]) << ", %rax\n";
            out_ << "  movq %rbp, %rsp\n  popq %rbp\n  ret\n";
            break;
    }
}

std::string CodeGen::generate(const ir::Function& fn) {
    out_ << ".globl " << fn.name << "\n" << fn.name << ":\n";
    out_ << "  pushq %rbp\n  movq %rsp, %rbp\n";
    out_ << "  subq $256, %rsp\n"; // fixed-size frame; real codegen sizes this

    for (auto& block : fn.blocks) {
        out_ << "." << block.label << ":\n";
        for (auto& instr : block.instrs) emitInstr(instr);
    }
    return out_.str();
}
```

```
} // namespace lumen
```

The **calling convention** shows up in exactly two lines: `pushq %rbp; movq %rsp, %rbp` (prologue — save the caller's frame pointer, establish ours) and `movq %rbp, %rsp; popq %rbp; ret` (epilogue — tear it down). The System V AMD64 ABI also says the return value goes in `%rax`, which is why `Return` moves its operand there before `ret`. This is the concrete, mechanical version of "how a function call actually works" from Phase 0/8's systems context.

6.3 Exercises

1. Trace the emitted assembly for `return 2 + 3;` by hand — confirm you understand every line, including why the constant round-trips through a stack slot at all (it doesn't need to — that inefficiency is exactly what a smarter codegen or a prior constant-folding pass removes).
2. **The real Phase-6 project:** replace the "every value gets a stack slot" strategy with linear-scan register allocation using the Phase 5 liveness results — assign the small number of live-at-once temps to actual registers (`%rax, %rbx, %rcx, %rdx, ...`), spilling to stack slots only when more values are live simultaneously than there are registers. This is the single biggest jump in code quality in the whole project.
3. **Debugging exercise:** the fixed `subq $256, %rsp` frame silently corrupts memory if a function ever needs more than 32 stack slots. Fix `generate()` to size the frame from `slots_.size()` instead of a hardcoded constant.
4. **Interview-style:** why does `idivq` need `cqto` before it but `imulq` doesn't need anything equivalent? (Hint: look up what `idivq` actually divides — a 128-bit dividend held in `%rdx:%rax` — versus what `imulq %rcx, %rax` computes.)

6.4 Real compiler comparison

LLVM's backend does instruction selection via pattern-matching **DAGs** (each basic block's instructions form a small dependency graph, matched against target-specific patterns — e.g. recognizing `mul` immediately followed by `add` as a single fused instruction on architectures that support it), then register allocation via graph coloring (`RegAllocGreedy`), then instruction scheduling for pipeline efficiency — three distinct passes where Lumen's codegen collapses everything into one direct IR-to-asm walk. GCC's backend (RTL — register transfer language) follows a similar multi-pass shape. The stack-slot approach above is closest to what an unoptimized `-O0` build produces from any real compiler — which is precisely why `-O0` code is famously slow: it's making the same simplifying choice we just made, on purpose, for fast compile times over fast runtime.

Phase 7 — Linking, Loading, and the OS boundary

Our codegen emits *assembly text*; turning that into a runnable program is not "the compiler" in the narrow sense but is essential to understand:

- **Assembler**: text asm → object file (machine code + a symbol table + relocation info)

- **Object format (ELF on Linux, PE on Windows)**: sections (`.text`, `.data`, `.bss`), symbol table, relocations (placeholders for addresses not known until link time)
- **Linker**: merges object files + libraries, resolves symbol references, applies relocations, produces an executable (or leaves dynamic symbols for the loader)
- **Loader** (OS-level, at `exec()` time): maps the executable into virtual memory, resolves any remaining dynamic-library symbols, sets up the initial stack, jumps to `_start`

For Lumen, we shell out to the system assembler/linker rather than writing our own — same as most teaching compilers and even some production ones for their non-primary targets — but we trace through exactly what those tools did so it isn't a black box.

7.2 Wiring `main.cpp` into the full pipeline

```cpp
```

```cpp
#include "lexer/lexer.hpp"
#include "parser/parser.hpp"
#include "sema/sema.hpp"
#include "ir/irgen.hpp"
#include "ir/optimize.hpp"
#include "codegen/codegen.hpp"
#include <fstream>
#include <iostream>

int main(int argc, char** argv) {
    if (argc < 2) { std::cerr << "usage: lumen <source-file>\n"; return 1; }

    std::ifstream in(argv[1]);
    std::ostringstream buf; buf << in.rdbuf();
    std::string source = buf.str();

    lumen::Lexer lexer(source);
    auto tokens = lexer.tokenize();

    lumen::Parser parser(std::move(tokens));
    lumen::Program program;
    try {
        program = parser.parseProgram();
    } catch (const std::exception& e) {
        std::cerr << e.what() << "\n";
        return 1;
    }

    lumen::Sema sema;
    if (!sema.check(program)) return 1; // errors already printed

    std::ofstream asmOut("out.s");
    for (auto& fn : program.functions) {
        lumen::IRGen irgen;
        auto irFn = irgen.generate(fn);
        while (lumen::opt::constantFold(irFn)) {} // fold to a fixed point
        lumen::CodeGen codegen;
        asmOut << codegen.generate(irFn);
    }
    asmOut.close();

    // Hand off to the system toolchain for assembling + linking — this
    // is the literal Phase 7 boundary: our compiler's job ends at the
    // .s file, `cc` (as a thin driver over `as`/`ld`) does the rest.
```

```
    return system("cc -no-pie out.s -o a.out");
}
```

`cc -no-pie out.s -o a.out` runs the **assembler** (`as`, text asm → ELF object file with `.text`, symbol table, relocations) then the **linker** (`ld`, pulls in the C runtime startup code that calls our `main`-equivalent, resolves any remaining symbols, produces the final ELF executable). `-no-pie` keeps the addresses simple for now (position-independent executables add a layer of indirection worth understanding *after* the basic model is solid — Exercise 7.4.3 below).

7.3 Seeing what the toolchain actually did

```
as out.s -o out.o
readelf -S out.o        # section headers: .text, .data, .bss, .symtab...
readelf -s out.o        # the symbol table — find your function name
objdump -d a.out         # disassemble the final linked executable
```

Running `objdump -d` on `a.out` and finding your function's assembly *exactly matches* `out.s` (modulo addresses) is the moment the whole pipeline stops being abstract — you're looking at the literal bytes the CPU executes, and you wrote every stage that produced them.

7.4 Exercises

1. Compile a Lumen program, then run the `readelf`/`objdump` commands above. Identify your function's address and confirm `_start` (the real entry point, not your function) is what the OS actually jumps to first — trace how it gets from there to calling your code.
2. Compile with and without `-static`; compare binary size and `ldd a.out` output (or `otool -L` on macOS) — this is the static vs dynamic linking trade-off made concrete.
3. **Research exercise:** what does `-no-pie` actually disable, and why do modern systems default to PIE executables? (Hint: ASLR — address space layout randomization — a security mitigation that depends on position-independent code.)
4. **Interview-style:** the linker resolves symbol *references* against symbol *definitions* across object files. What error do you get if a function is declared but never defined anywhere being linked, and at which stage (compile vs assemble vs link) does that error actually surface? Why not earlier?

Phase 8 — Systems context (woven throughout; reference)

CPU execution model, registers, cache-awareness, stack vs heap, function-call mechanics, runtime libraries, syscalls — these get taught **in situ** as they become relevant (register allocation pulls in registers/ABI; codegen pulls in stack frames; linking pulls in ELF/loader) rather than as a separate detached module, since they land much better with a concrete reason to care about them.

Phase 9 — Extensions (after the core compiler works end-to-end)

9.1 Retarget to LLVM IR

Instead of `CodeGen` emitting x86-64 text directly, emit **LLVM IR text** from the same `ir::Function` and hand it to `llc`/`clang` for instruction selection, register allocation, and optimization. The mapping is close to 1:1 with what we already built:

```llvm
define i64 @main() {
entry:
  %t1 = add i64 2, 3
  ret i64 %t1
}
```

Notice LLVM IR is **already SSA** — every `%tN` assigned exactly once — which is why Exercise 4.4.3 (hand-converting our TAC to SSA) is the exact skill this retarget needs. Compare the assembly LLVM's backend produces (`llc out.ll -o out.s`) against Lumen's own Phase 6 output for the same program: LLVM's will use real registers, fuse instructions, and likely be a third the length — a direct, visible payoff for register allocation (the Phase 6.3 Exercise 2 you may or may not have done yet).

9.2 A minimal JIT

Skip the file-based executable entirely:

1. `mmap` a page of memory with `PROT_READ | PROT_WRITE | PROT_EXEC`.
2. Assemble machine code bytes directly into that page (either by hand-encoding a tiny subset of x86-64, or by shelling out to `as`/`objcopy` to get raw bytes and copying them in — the pragmatic version).
3. Cast the page's address to a function pointer (`auto fn = (int(*)())page;`) and call it.

This is the mechanism (not the algorithm) behind every JIT — V8, the JVM's C2, LuaJIT. The "algorithm" difference is *when* they decide to JIT (typically: after profiling shows a function is hot), which Lumen's minimal version skips by JITing everything immediately.

9.3 Garbage collection (conceptual)

Only relevant once Lumen has heap-allocated data (arrays, structs) — not needed for the int-only core. Mark-sweep in one paragraph: maintain a set of GC "roots" (stack variables, registers currently holding heap pointers); **mark** phase walks from roots, following every pointer, marking each reachable object; **sweep** phase walks the whole heap, freeing anything unmarked. The entire field of GC design is about doing this without pausing the program for too long (generational, incremental, concurrent collectors) — worth knowing conceptually even without implementing it for Lumen's current feature set.

9.4 Add a real feature end-to-end: closures

Pick this as your first full-pipeline feature exercise once Phases 0–7 are solid, because it touches *every* phase and forces you to see how they interlock:

- **Lexer/Parser:** no new tokens needed if you use existing function syntax; new AST node for a function-as-value.
- **Sema:** must now resolve free variables (names used inside the closure but declared outside it) — this is where "capture" is decided.
- **IR gen:** a closure lowers to (a) a code pointer plus (b) an environment struct holding captured variables' values — two things bundled together, not one.
- **Codegen:** calling a closure means loading the environment pointer into a register the callee's prologue knows to expect (this is exactly the ABI-convention concept from Phase 6, applied to a new case).

Working through this one feature, tracing it through every phase you've already built, is worth more than reading about ten more features in the abstract.

Where we are

Complete, in this document, today: the full pipeline, Phase 0 through Phase 9 — a working lexer, recursive-descent parser, AST, semantic analyzer, IR generator, constant folder, liveness analysis, and x86-64 codegen, wired together into a compiler that takes a `.lum` file to a running executable, plus concrete extension paths (LLVM retarget, JIT, GC, closures) once the core is solid.

What's deliberately left as your work, not mine: every exercise throughout — most importantly Phase 3 Exercise 3 (the visitor-pattern refactor) and Phase 6 Exercise 2 (real register allocation), which are the two places the "textbook toy" version above turns into something closer to a real compiler's internals. I gave you working code everywhere so you have a correct baseline to diff against, not so you can skip typing it — build each phase yourself from the explanation before checking it against what's here.

Suggested order: build Phase 0–1 first and get the lexer passing your own test snippets, then work phase by phase, doing the exercises as you go rather than after. Bring me your code, your test failures, and your answers to the interview-style questions — that's where the actual mentoring happens from here, not in more prose.